



PRÉ-RENTRÉE 2026-2027

Séance 3 - Cahier élève

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ distinguer paramètre et argument ;
- ☐ distinguer print, return et None ;
- ☐ comprendre la portée locale ;
- ☐ documenter une fonction ;
- ☐ écrire des assertions de test ;

Rituel sans ordinateur

Question 1

Que vaut `r` après l'exécution ?

```
def double(x):  
    print(2 * x)  
  
r = double(5)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Dans `def aire_disque(rayon):`, distinguer le paramètre, l'argument dans `aire_disque(3)` et la valeur renvoyée.

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

```
def carre(n: int) -> int:
    """Renvoie le carré de n."""
    return n * n
```

- `n` est un paramètre ;
- dans `carre(5)`, `5` est l'argument ;
- `return` termine la fonction et transmet une valeur ;
- `print` affiche mais ne renvoie pas cette valeur ;
- sans `return`, Python renvoie `None` ;
- les variables créées dans la fonction sont locales.

Contrat

- précondition : ce qui doit être vrai avant l'appel ;
- postcondition : ce qui est garanti après l'appel ;
- `assert` peut vérifier une condition ;
- un jeu de tests ne prouve pas l'absence de bugs, mais il réduit le risque.

Activité commune - prédire les retours

```
def f(x):
    print(x + 1)

resultat = f(4)
```

1. Qu'est-ce qui est affiché ?
2. Que vaut `resultat` ?
3. Comment modifier `f` pour renvoyer 5 ?

Conception de tests

Pour une fonction `maximum_deux(a, b)`, proposer :

- un cas ordinaire ;
- un cas où `a == b` ;
- un cas avec deux valeurs négatives.

Parcours Fondations - Ahmad

1. Écrire `est_pair(n)`.
2. Écrire `maximum_deux(a, b)`.
3. Écrire `compte_occurrences(valeurs, cible)`.
4. Ajouter une docstring à chaque fonction.
5. Écrire deux assertions par fonction.

Parcours Fiabilisation - Ahmed

1. Écrire une précondition pour une fonction de moyenne.
2. Choisir le comportement sur liste vide : assertion ou `None`, et le justifier.
3. Écrire des tests limites et invalides.
4. Corriger une fonction qui modifie une variable globale.
5. Lire la documentation de `math.isclose` et l'utiliser pour tester un flottant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :